Assignment 11 – Shadow Mapping

Overview

The goal of this assignment is to implement real-time shadows with shadow mapping. At a minimum, your engine must support a single shadow-casting directional light.

Task Overview

Here is the high-level overview of tasks. Some of these are detailed below, but the majority of the implementation details can be found in the associated Shadow Mapping reading on MyCourses:

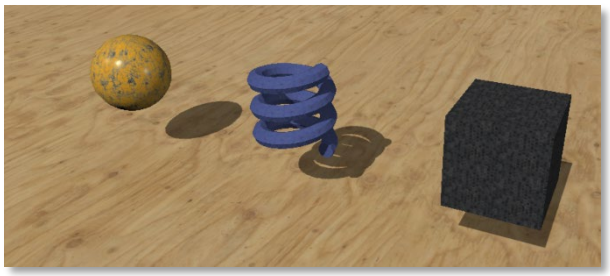
- ❑ **Move and/or create new entities** in your scene to appropriately showcase shadows
- ❑ Create all necessary **shadow mapping data and resources** (and any helper functions you'd like)
- ❑ Properly **render to the shadow map** at the beginning of each frame
- ❑ **Utilize the shadow map** in your existing shaders to determine if a pixel is in shadow
- ❑ Ensure shadows are **only applied to the corresponding light** (instead of all lights)
- ❑ Ensure you have no **warnings**, **memory leaks** or **DX resource leaks**

Task Details

Some of the task details are outlined below. Specific implementation details can be found in the associated Shadow Mapping reading on MyCourses.

Appropriately Showcasing Shadows

Your project probably has a set of meshes floating in space. While this has been fine for previous assignments, shadows require a more purposeful arrangement of entities to properly showcase.



A simple example scene from my shadow demo can be found to the right. The three floating objects move up and down over time.

1. Create a **floor** or other large surface below all of your objects. This will serve as an obvious shadow-receiving entity. This can simply be a cube that has been scaled up on the X and Z axes.
2. Ensure the floor's material is relatively **light in color** to easily distinguish shadows.
3. Ensure you have multiple objects **above and/or intersecting** with the floor. These will serve as your primary shadow-casting entities.
4. Have some of your entities **translate and/or rotate** over time to test the real-time nature of your shadows.

Shadow Mapping Data and Resources

As noted in the Shadow Mapping reading, you'll need the following resources by the end of your implementation. You'll most likely just declare these in your Game.h file. Specifics on each can be found in the associated reading.

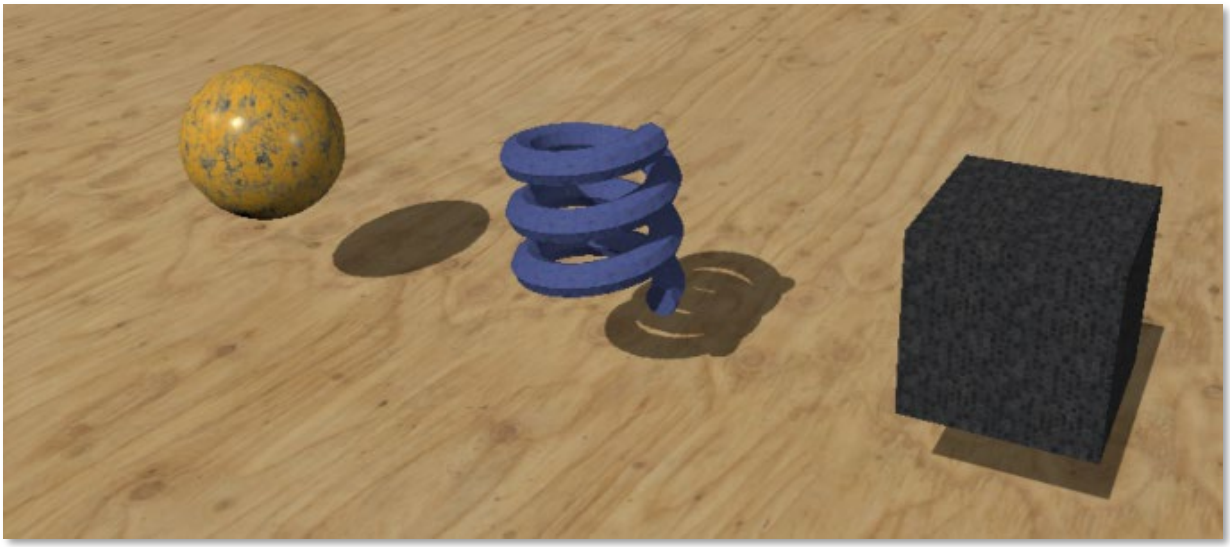
```
Microsoft::WRL::ComPtr<ID3D11DepthStencilView> shadowDSV;
Microsoft::WRL::ComPtr<ID3D11ShaderResourceView> shadowSRV;
Microsoft::WRL::ComPtr<ID3D11RasterizerState> shadowRasterizer;
Microsoft::WRL::ComPtr<ID3D11SamplerState> shadowSampler;
DirectX::XMFLOAT4X4 shadowViewMatrix;
DirectX::XMFLOAT4X4 shadowProjectionMatrix;
```

Helper Functions

There are several overarching tasks for shadow mapping, such as resource creation and rendering the scene to the shadow map. While not required, each of these would make a good candidate for a helper function to clean up your overall initialization and draw steps.

Reaching Out for Help

The rest of this assignment is up to you. If you get stuck, or if a part of the implementation is not clear, please reach out for help!

Example Output

Below you'll find an example of properly implemented shadows. Notice that you can still see details of the surface (the wooden floor) within the shadows. Only the primary directional light causes shadows even though there are multiple lights in the scene. Your results should be similar.